

```
#GCD of two numbers
x, y = input("Enter 2 Number: ").split()
x = int(x); y = int(y)

#x, y = [int(a) for a in input().split()]

#x, y = map(int, input().split())

if (y > x):
    x, y = y, x
while(y>0):
    x, y = y, x % y # 50, 40 --> 40, 10 --> 10, 0
print('GCD is: ', x)
```

```
Enter 2 Number: 56 32
GCD is: 8
```

```
e = [int(a) for a in input().split()]
e
```

```
12 55 45 78
[12, 55, 45, 78]
```

✓ match Statement

“Switch Case in Python” called “Python Structure Pattern Matching”

match subject:

```
case <pattern_1>:
```

```
    <action_1>
```

```
case <pattern_2>:
```

```
    <action_2>
```

```
case <pattern_3>:
```

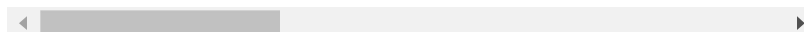
```
    <action_3>
```

```
case _:
```

```
    <action_wildcard>
```

```
!sudo apt-get install python3.10
!sudo update-alternatives --install /usr/bin/python3 pyth
!sudo update-alternatives --set python3 /usr/bin/python3.

Reading package lists... Done
Building dependency tree
Reading state information... Done
The following package was automatically installed and
  libnvidia-common-460
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  libpython3.10-minimal libpython3.10-stdlib python3.
Suggested packages:
  python3.10-venv binfmt-support
The following NEW packages will be installed:
  libpython3.10-minimal libpython3.10-stdlib python3.
0 upgraded, 4 newly installed, 0 to remove and 5 not
Need to get 5,098 kB of archives.
After this operation, 19.5 MB of additional disk spac
Get:1 http://ppa.launchpad.net/deadsnakes/ppa/ubuntu
Get:2 http://ppa.launchpad.net/deadsnakes/ppa/ubuntu
Get:3 http://ppa.launchpad.net/deadsnakes/ppa/ubuntu
Get:4 http://ppa.launchpad.net/deadsnakes/ppa/ubuntu
Fetched 5,098 kB in 0s (13.4 MB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed,
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (This frontend requires a controlling tty.)
debconf: falling back to frontend: Teletype
dpkg-preconfigure: unable to re-open stdin:
Selecting previously unselected package libpython3.10
(Reading database ... 123991 files and directories cu
Preparing to unpack .../libpython3.10-minimal_3.10.8-
Unpacking libpython3.10-minimal:amd64 (3.10.8-1+bioni
Selecting previously unselected package python3.10-mi
Preparing to unpack .../python3.10-minimal_3.10.8-1+b
Unpacking python3.10-minimal (3.10.8-1+bionic1) ...
Selecting previously unselected package libpython3.10
Preparing to unpack .../libpython3.10-stdlib_3.10.8-1
Unpacking libpython3.10-stdlib:amd64 (3.10.8-1+bionic
Selecting previously unselected package python3.10.
Preparing to unpack .../python3.10_3.10.8-1+bionic1_a
Unpacking python3.10 (3.10.8-1+bionic1) ...
Setting up libpython3.10-minimal:amd64 (3.10.8-1+bion
Setting up libpython3.10-stdlib:amd64 (3.10.8-1+bioni
Setting up python3.10-minimal (3.10.8-1+bionic1) ...
Setting up python3.10 (3.10.8-1+bionic1) ...
Processing triggers for man-db (2.8.3-2ubuntu0.1) ...
Processing triggers for mime-support (3.60ubuntu1) ..
update-alternatives: using /usr/bin/python3.10 to pro
```



```
col=input('enter color(r/g/b):')
print('Color is:', end="")
match col:
    case 'r':
        print('RED')
    case 'g':
        print('Green')
    case 'b':
        print('Blue')
    case _:
        print('invalid input')

        enter color(r/g/b):r
        Color is:RED
```

```
!python --version
```

```
Python 3.7.15
```

```
conda update python
```

```
conda install python=3.10
```

1. Write a program to Count the total number of digits in given integer number
2. Write a program to find first and last digit of a given integer number.
3. Write a program to swap first and last digits of a given integer number.
4. Generate a mirror image of a given integer number (or Reverse a given integer number)
5. Check whether a given integer number is palindrome or not
6. Write a program to convert a decimal number into binary
7. Write a program to convert a binary number into decimal number
8. Write a program to convert a decimal number to hexadecimal. Write a program to convert a decimal number to octal.
9. Write a program to find LCM (least common multiplier) of any two numbers
10. Write a program to print multiplication table of a given number

11. Write a program to print multiplication table for numbers given in a range
12. Write a program to display all prime numbers within a range
13. Write a program to calculate the sum of series up to n term. For example, if n =5 the series will become $2 + 22 + 222 + 2222 + 22222 = 24690$

```
a = input("Enter a Name: ").lower()
b = int(len(a)/2)
flag = 1
for i in range(b+1):
    if a[i]!=a[-(i+1)]:
        flag = 0;
        break
if(flag == 1):
    print("{} is a Palindrome".format(a))
else:
    print("{} is Not a Palindrome".format(a))
```

```
Enter a Name: Nitin
nitin is a Palindrome
```

```
def isPalindrome(s):
    s = s.lower()
    l = len(s)
    if l < 2:
        return True
    elif s[0] == s[l - 1]:
        return isPalindrome(s[1: l - 1])
    else:
        return False
```

✓ Functions

```
def func_name(par1, par2,...):
```

```
    body of code
```

```
#addition
def my_add(x, y):
    return x+y

sum=my_add(3.5, 8)
print(sum)
```

```
11.5
```

```
#return multiple values
def power(input):
    return input**2,input**3
```

```
power(3)
```

```
(9, 27)
```

```
def grade(marks):
    """Compute Grade of Students"""
    if marks > 90:
        print("A")
    elif marks > 70:
        print("B")
    elif marks > 50:
        print("C")
    else:
        print("D")
mark = int(input("Enter your marks"))
grade(mark)
```

```
Enter your marks78
B
```

```
def grade(marks):
    """Compute Grade of Students"""
    if marks > 90:
        print("A")
    elif marks > 70:
        print("B")
    elif marks > 50:
        print("C")
    else:
        print("D")
grade(90)
```

```
B
```

```
help(print)
```

```
Help on built-in function print in module builtins:
```

```
print(...)
    print(value, ..., sep=' ', end='\n', file=sys.std
```

Prints the values to a stream, or to sys.stdout b
 Optional keyword arguments:
 file: a file-like object (stream); defaults to t
 sep: string inserted between values, default a
 end: string appended after the last value, defa
 flush: whether to forcibly flush the stream.

```
def SumofDigits(num):
    sum=0
    while(num!=0):
        sum = sum + num%10
        num=num//10
    return sum
```

```
SumofDigits(1009)
```

```
10
```

```
help(grade)
help(SumofDigits) # implies you're seeking help or docum
```

```
Help on function grade in module __main__:
```

```
grade(marks)
    Compute Grade of Students
```

```
Help on function SumofDigits in module __main__:
```

```
SumofDigits(num)
```

Help on function grade in module **main**:", indicates that the documentation displayed is for the function named grade that is defined in the current module, which is typically the main script you are working with

```
help(print)
```

```
Help on built-in function print in module builtins:
```

```
print(...)
    print(value, ..., sep=' ', end='\n', file=sys.std
```

Prints the values to a stream, or to sys.stdout b

Optional keyword arguments:

file: a file-like object (stream); defaults to t
 sep: string inserted between values, default a
 end: string appended after the last value, defa
 flush: whether to forcibly flush the stream.

grade.__doc__ #refers to the documentation string (docs
 #It provides a way to document the purpose, usage, and pa

'Compute Grade of Students'

```
def fibb(num):
    sum =0
    n1,n2=0,1
    while num > sum:
        print(sum)
        n1 =n2
        n2 = sum
        sum =n1+n2
num=int(input("enter your range:"))
fibb(num)
```

```
enter your range:5
0
1
1
2
3
```

```
def PRINTNAME():
    print("SIT")
    return 10, "SIT", 11.23
print(PRINTNAME())
```

```
SIT
(10, 'SIT', 11.23)
```

```
def rev(p):
    x=0
    while p!=0:
        x=x*10+p%10
        p=p//10
    print(x)
rev(123)
```

```
321
```

```
def fibo(n):    # write Fibonacci series up to n
    """Print a Fibonacci series up to n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

fibo(10)
```

0 1 1 2 3 5 8

```
def countvowel(name):
    count=0
    size=len(name)
    print(size)
    vow=['a','e','i','o','u']
    for i in range(0,size):
        if(name[i] in vow):
            count+=1
    print(count)

name=input()
countvowel(name)
```

riya
4
2

With return

```
def grade(marks):
    if marks > 90:
        return("A")
    elif marks > 70:
        return("B")
    elif marks > 50:
        return("C")
    else:
        return("D")
marks = int(input("Enter your marks : "))
print(f"Your grade is : {grade(marks)}")
```

Enter your marks : 6
Your grade is : D


```

lines = 5
for i in range(lines,0,-1):
    for j in range(0, i ):
        print("#", end=' ')
    print("\r")

```

```

# # # # #
# # # #
# # #
# #
#

```

```

x = 10
id(x)

```

```

11126976

```

```

def f(x):
    y = 5
    print(f'inside  function y is: {y} & id:{id(y)}')

```

```

x = 5
f(x)
print(f'outside function x is: {x}  & id:{id(x)}')

```

```

inside  function y is: 5 & id:11126816
outside function x is: 5  & id:11126816

```

```

x= ['Nagpur', 'Pune', 'Mumbai', 'Nasik']
print(f'x is: {x} & id:{id(x)}')
city_list = ['Nagpur', 'Pune', 'Mumbai', 'Nasik']
print(f'City list is: {city_list} & id:{id(city_list)}')

```

```

x is: ['Nagpur', 'Pune', 'Mumbai', 'Nasik'] & id:1405
City list is: ['Nagpur', 'Pune', 'Mumbai', 'Nasik'] &

```



```

def f(x):
    x.insert(0,5)
    x.insert(2,15)
    x.insert(4,25)
    x.insert(6,35)
    print(f'updated list is:{x} {id(x)}')

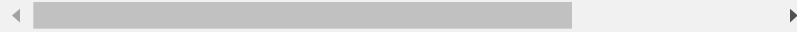
```

```

num_list=[10,20,30,40]
print(f'original list is: {num_list}')
f(num_list)
print(f'outside function list is :{num_list} {id(num_list)}')

```

```
original list is: [10, 20, 30, 40]
updated list is:[5, 10, 15, 20, 25, 30, 35, 40] 14050
outside function list is :[5, 10, 15, 20, 25, 30, 35,
```



```
def f(x):
    x= ['Nagpur', 'Pune', 'Mumbai', 'Nasik']

    print(f'inside  function city_list is: {x} & id:{id(x)}')

city_list = ['Nagpur', 'Pune', 'Mumbai', 'Nasik']
print(f'Pre outside function city_list is: {city_list} & id:{id(city_list)}')
f(city_list)
print(f'outside function city_list is: {city_list} & id:{id(city_list)}')
```

```
Pre outside function city_list is: ['Nagpur', 'Pune', 'Mumbai', 'Nasik']
inside  function city_list is: ['Nagpur', 'Pune', 'Mumbai', 'Nasik']
outside function city_list is: ['Nagpur', 'Pune', 'Mumbai', 'Nasik']
```



```
def count(n):
    c=0
    while(n!=0):
        n//=10
        c+=1
    return c
count(1234)
```

```
4
```

```
def f(lst, i, ele):
    lst.insert(i, ele)
    #lst[i] = ele

num_lst=[10,20,30,40]
print(f'original list is: {num_lst}')
ch = 'Y'
while(ch == 'Y'):
    val = int(input("Enter Element: "))
    pos = int(input("Enter Position: "))
    f(num_lst, pos-1, val)
    ch = input("Do you want to add more elements(Y/N): ")

print(f'outside function list is :{num_lst}')
```

```
original list is: [10, 20, 30, 40]
Enter Element: 20
Enter Position: 3
Do you want to add more elements(Y/N): Y
Enter Element: 50
```

```

Enter Position: 5
Do you want to add more elements(Y/N): N
outside function list is :[10, 20, 20, 30, 50, 40]

```

```

def f(x):
    x = '---'
    print(f'inside  function city_list is: {x} & id: {id(

city_list = ['Nagpur', 'Pune', 'Mumbai', 'Nasik']
f(city_list)
print(f'outside function city_list is: {city_list} & id:

    inside  function city_list is: --- & id: 140466811450
    outside function city_list is: ['Nagpur', 'Pune', 'Mu

```



```

def pow(x, y=2):
    return x**y

print (f"The pow(4) is {pow(4)}")
print(f"The pow(5,3) is {pow(5,3)}")
print (f"The pow(5) is {pow(5)}")

    The pow(4) is 16
    The pow(5,3) is 125
    The pow(5) is 25

def sup_pow(x, y=2, z=2):
    return(x**y)**z
print(f'Super pow (2,2,2) is {sup_pow(2)}')
print(f'Super pow (2,3,2) is {sup_pow(2,3)}')
print(f'Super pow(2,3,3) is {sup_pow(2,3,3)}')

    Super pow (2,2,2) is 16
    Super pow (2,3,2) is 64
    Super pow(2,3,3) is 512

```

```

def fact(n=5):
    f=1
    while(n>0):
        f*=n
        n=n-1
    return f

```

```

fact(7)
fact()

```

```

120

```

```
def f(a,L=[]):
    L.append(a)
    print(id(L),end=' : ')
    return L

print(f(1))
print(f(2))
print(f(3))

140323008534592 : [1]
140323008534592 : [1, 2]
140323008534592 : [1, 2, 3]
```

```
def f(L):
    print(id(L))
    L.append(7)
    print(id(L))
    print(L[2:4])
```

```
L=[1,2,3,4,5]
f(L)

132663536740352
132663536740352
[3, 4]
```

```
def f1(a, L=None):
    if L is None:
        L=[]
    L.append(a)
    print(id(L), end=' : ')
    return L
```

```
print(f1(1,None))
print(f1(2))
print(f1(3))

139964882810176 : [1]
139964882809920 : [2]
139964882809600 : [3]
```

KEYWORD ARGUMENT

In Python, a keyword argument is an argument passed to a function or method preceded by a keyword and an equals sign. When calling the function, you specify the name of the argument along with its value. This allows you to provide

arguments to a function in any order, as long as you use the keyword associated with each argument.

```
def student(college, name, pass_year=2026):
    print("college name : ", college)
    print("name of the student : ", name)
    print("passing year is :",pass_year)
```

```
student(name="Priya",college="SIT")
```

```
college name :  SIT
name of the student :  Priya
passing year is : 2026
```

```
def amt(qty,price,discount=0):
    t=qty*price
    return(t-(t*(discount/100)))
#amt(6,discount=10,price=5)
#amt(6,price=5)
#amt(6,5,10)
#amt()
#amt(discount=10,price=5,6)
#amt(qty=6,5,discount=10)
#amt(6,qty=10,price=5)
#amt(6,dis=10,price=5)
amt(price=5,qty=6)
```

```
30.0
```

#Arbitrary Arguments

```
def fun(*arg_list):          #*arg_list syntax collects all
    for arg in arg_list:
        print(arg, end='\n')
    print(arg_list)
```

```
fun("College Name","Branch","Students")
```

```
College Name
Branch
Students
('College Name', 'Branch', 'Students')
```

```
def college(name,**data):    ***data parameter, collects
    print(name)
    print(data)
```

```
college('SIT',streetno=21,city='Nagpur',pincode=121348)
```

```
SIT
{'streetno': 21, 'city': 'Nagpur', 'pincode': 121348}
```

✓ Lambda Function

A lambda function is a small anonymous function.

A lambda function can take any number of arguments, but can only have one expression.

a shorthand notation if you're too lazy to define a function

Syntax:

lambda arguments : expression

```
x = lambda a : a + 10  
print(x(5))
```

15

```
(lambda a : a + 10) (5)
```

15

```
x = lambda a, b, c : a + b * c  
print(x(5, 6, 2))  
print(x)
```

17

<function <lambda> at 0x7f31f06e12d0>

✓ Why Use Lambda Functions?

```
#Line of code can be reduced using lambda function  
#Lambda is not stored in RAM and therefore it saves RAM m  
def average(x, y):  
    return (x + y)/2  
print(average(4, 3))
```

3.5

```
print((lambda x, y: (x + y)/2)(4, 3))
```

3.5

```
a=lambda x,y: x+y
print(a(5,10))
```

```
15
```

```
8 % 2 and 'odd' or 'even'
```

```
'even'
```

```
'even' and 'odd'
```

```
'odd'
```

```
(lambda x:x%2 and 'odd' or 'even')(3)
```

```
'odd'
```

```
(lambda x:
(x % 2 and 'odd' or 'even'))(3) #Multiline single expres
```

```
'odd'
```

Python lambda expressions support all the different ways of passing arguments

- Positional arguments
- Named arguments (sometimes called keyword arguments)
- Variable list of arguments (often referred to as varargs)
- Variable list of keyword arguments
- Keyword-only arguments

```
print((lambda x, y, z: x + y + z)(1, 2, 3))
print((lambda x, y, z=3: x + y + z)(1, 2))
print((lambda x, y, z=3: x + y + z)(1, y=2))
print((lambda *args: sum(args))(1,2,3)) #This defines an
#and returns the sum of those arguments using the built-i
print((lambda **kwargs: sum(kwargs.values()))(one=1, two=
#This defines an anonymous function (lambda function) tha
#Inside the function, it calculates the sum of the values
print((lambda x, *, y=0, z=0: x + y + z)(1, y=2, z=3))
#The * before y indicates that y must be passed as a keyw
```

```
6
```

```
6
```

```
6
```

6
6
6

The filter() method filters the given sequence with the help of a function that tests each element in the sequence to be true or not.

Syntax: filter(function, sequence)

Parameters: function: function that tests if each element of a sequence true or not.

sequence: sequence which needs to be filtered, it can be sets, lists, tuples, or containers of any iterators.

Returns: returns an iterator that is already filtered.

```
num=[1,2,3,4,5,6,7,8]
even=list(filter(lambda x:x%2==0, num))
print(even)
odd=list(filter(lambda x:x%2!=0, num))
print(odd)
```

```
[2, 4, 6, 8]
[1, 3, 5, 7]
```

```
date=['15/1/2022', '3/11/2019', '20/12/2016']
A=list(filter((lambda d:int(d.split('/')[2])>2016),date))
print(A)
```

```
['15/1/2022', '3/11/2019']
```

```
dt='1/11/1111'
dt.split('/') [0]
```

```
'1'
```



```

num = [[5, 6, 'Sharvil'], [7, 8, 'Priya'], [6, 9, 'Nupur']]

def my_avg(x):
    if ((x[0]+x[1])/2) > 7:
        return 'PASS'
    else:
        return 'FAIL'

P = list(map(my_avg, num))
print(type(P))
print(P)
j=0

for i in num:
    i.append(P[j])
    j=j+1

print(num)

<class 'list'>
['FAIL', 'PASS', 'PASS']
[[5, 6, 'Sharvil', 'FAIL'], [7, 8, 'Priya', 'PASS'],

```

List Comprehension List comprehension is a concise way of creating lists in Python. It allows you to generate lists using a single line of code, often without the need for explicit loops. The basic syntax for list comprehension is: **[expression for item in iterable if condition]**, where **expression** is the operation performed on each item. **item** is the variable representing elements in the iterable. **iterable** is any iterable Python object, like lists, tuples, strings, etc. **condition (optional)** is an expression that filters items.

```

# Squares of numbers from 0 to 9 using a loop
squares = []
for i in range(10):
    squares.append(i ** 2)
print(squares)

```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```

# Squares of numbers from 0 to 9 using list comprehension
squares = [i ** 2 for i in range(10)]
print(squares)

```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
# Even squares of numbers from 0 to 9 using list comprehension
even_squares = [i ** 2 for i in range(10) if i % 2 == 0]
print(even_squares)
```

```
[0, 4, 16, 36, 64]
```

✓ map Function

map() function returns a map object(which is an iterator) of the results after applying the given function to each item of a given iterable (list, tuple etc.)

Syntax:

```
map(fun, iter)
```

Parameters:

fun : It is a function to which map passes each element of given iterable. iter : It is a iterable which is to be mapped.

NOTE : You can pass one or more iterable to the map() function.

Returns:

Returns a list of the results after applying the given function to each item of a given iterable (list, tuple etc.)

NOTE : The returned value from map() (map object) then can be passed to functions like list() (to create a list), set() (to create a set) .

```
x, y = map(int,input().split())
```

```
5 6
```

```
type(x)
```

```
int
```

```
y = list(map(int, input().split()))
print(y)

5 3 4 9
[5, 3, 4, 9]

l=[45,32,-10,88,12,-21,13]
pn=list(filter((lambda x:x>=0),l))
print(pn)
def f1(i):
    if i%2==0:
        return('EVEN')
    else:
        return('ODD')
oddeven=list(map(f1,pn))
print(oddeven)

#oddeven=list(map(lambda x:'EVEN' if x%2==0 else 'ODD',pn)
#print(oddeven)

[45, 32, 88, 12, 13]
['ODD', 'EVEN', 'EVEN', 'EVEN', 'ODD']

def addition(x):
    return x+x

# Return double of n

# double all numbers using map()
numbers = (1, 2, 3, 4)
result = map(addition, numbers)
print(type(result))
print(list(result))

<class 'map'>
[2, 4, 6, 8]

# Return square of n
def sq(n):
    return n * n

# double all numbers using map()
numbers = {1, 2, 3, 4}
result = map(sq, numbers)
print(list(result))

[1, 4, 9, 16]
```

```

# Add two lists using map and lambda

numbers1 = [1, 2, 3]
numbers2 = [4, 5, 6]

result = map(lambda x, y: x + y, numbers1, numbers2)
print(list(result))

    [5, 7, 9]

# Using Multiple Iterables with Python map()
lst = [1, 2, 3, 4, 5]
mod_lst = [10, 20, 30, 40, 50]

Mul_Res = list(map(lambda x, y: x*y, lst, mod_lst))

print(Mul_Res)

    [10, 40, 90, 160, 250]

# List of strings
l = ['sat', 'bat', 'cat', 'mat']

# map() can listify the list of strings individually
test = list(map(list, l))
print(test)

    [['s', 'a', 't'], ['b', 'a', 't'], ['c', 'a', 't'], [
    <img alt="Horizontal scrollbar" data-bbox="115 575 613 590"/>
]

nums = list(map(int, input().split()))
def fact(x):
    j = 1
    for i in range(1, x+1):
        j = j*i
    return j
fct = list(map(fact, nums))
print(fct)

    5 6 7
    [120, 720, 5040]

date=['15/1/2022', '3/11/2019', '20/12/2016']
A=list(filter((lambda d:int(d.split('/')[2])>2016),date))
print(A)

B=list(map(lambda x: int(x.split('/')[2]), A))
print(B)

```

```
['15/1/2022', '3/11/2019']  
[2022, 2019]
```

Reduce function

`reduce()` is defined in “functools” module. `reduce (fun, seq)` takes function as 1st and sequence as 2nd. We can use the `reduce()` function to reduce a sequence to a single output value by applying a function repeatedly in order to output a single value.

```
import functools  
num=[1,2,3,4]  
functools.reduce(lambda x,y: x+y, num)
```

10

Python Module

- A python script with python functions and statements
- Import a module before having access to all the functions within
- Most Python distributions come with plenty of build-in modules

✓ Python Modules

In Python, a module is a file containing Python definitions and statements. The file name is the module name with the suffix `.py` appended. Modules can contain functions, classes, and variables, and they allow you to organize your code into logical units that can be easily reused and imported into other Python programs.

- `math`, `sys`, `os`..
- `NumPy`, high performance in vector & matrix using vector computation
- `SciPy`: based on `Numpy`, including many scientific algorithms.

- pandas
- matplotlib, pyplot, pylab

Reference to [Python3.x standard libraries](#)

```
import dis #The dis module in Python is a built-in modul
#that is executed by the Python interpreter. The dis modu
#that make up your Python code.
```

```
#import a module
```

```
add = lambda x, y: x + y
print(type(add))
dis.dis(add)
print(add)
```

```
<class 'function'>
  4          0 LOAD_FAST          0 (x)
          2 LOAD_FAST          1 (y)
          4 BINARY_ADD
          6 RETURN_VALUE
<function <lambda> at 0x78a805ad6dd0>
```

```
def add(x, y): return x + y
print(type(add))
dis.dis(add) #This line disassembles the bytecode of the
#It prints the bytecode instructions that make up the fun
#Each bytecode instruction in Python has an associated by
print(add)
```

```
<class 'function'>
  1          0 LOAD_FAST          0 (x)
          2 LOAD_FAST          1 (y)
          4 BINARY_ADD
          6 RETURN_VALUE
<function add at 0x78a805ad7490>
```

```
#import a module
```

```
import math
print(math.pi)
print(math.sin(60))
print(math.sqrt(4))
```

```
3.141592653589793
-0.3048106211022167
2.0
```

```
from math import sqrt
sqrt(9)
```

3.0

```
print(math.pi)
```

3.141592653589793

```
dir(math)
```

```
#dir stands for "directory".
```

```
#It's a built-in function that returns the list of attrib
```

```
#get information of a module
```

```
['_doc_',  
 '__loader__',  
 '__name__',  
 '__package__',  
 '__spec__',  
 'acos',  
 'acosh',  
 'asin',  
 'asinh',  
 'atan',  
 'atan2',  
 'atanh',  
 'ceil',  
 'copysign',  
 'cos',  
 'cosh',  
 'degrees',  
 'e',  
 'erf',  
 'erfc',  
 'exp',  
 'expm1',  
 'fabs',  
 'factorial',  
 'floor',  
 'fmod',  
 'frexp',  
 'fsum',  
 'gamma',  
 'gcd',  
 'hypot',  
 'inf',  
 'isclose',  
 'isfinite',  
 'isinf',  
 'isnan',  
 'ldexp',  
 'lgamma',  
 'log',  
 'log10',  
 'log1p',  
 'log2',
```

```
'modf',
'nan',
'pi',
'pow',
'radians',
'remainder',
'sin',
'sinh',
'sqrt',
'tan',
'tanh',
'tau',
'trunc']
```

```
import math
nums = list(map(int, input().split()))
pst = list(map(math.factorial, nums))
print(pst)
```

```
6 7 8
[720, 5040, 40320]
```

```
#get information
help(math)
```

Help on built-in module math:

NAME

math

DESCRIPTION

This module provides access to the mathematical defined by the C standard.

FUNCTIONS

acos(x, /)

Return the arc cosine (measured in radians)

acosh(x, /)

Return the inverse hyperbolic cosine of x.

asin(x, /)

Return the arc sine (measured in radians) c

asinh(x, /)

Return the inverse hyperbolic sine of x.

atan(x, /)

Return the arc tangent (measured in radians

atan2(y, x, /)

Return the arc tangent (measured in radians

Unlike atan(y/x), the signs of both x and y


```
atanh(x, /)
    Return the inverse hyperbolic tangent of x.
```

```
ceil(x, /)
    Return the ceiling of x as an Integral.

    This is the smallest integer >= x.
```

```
copysign(x, y, /)
    Return a float with the magnitude (absolute value) of x and the
    sign of y. On platforms that support signed zeros, copysign(0.0, -0.0)
    returns -1.0.
```

```
cos(x, /)
    Return the cosine of x (measured in radians).
```

```
cosh(x, /)
    Return the hyperbolic cosine of x.
```

```
degrees(x, /)
    Convert angle x from radians to degrees.
```

```
erf(x, /)
    Error function at x.
```

```
erfc(x, /)
```

```
# get information of a module function
help(print)
```

Help on built-in function print in module builtins:

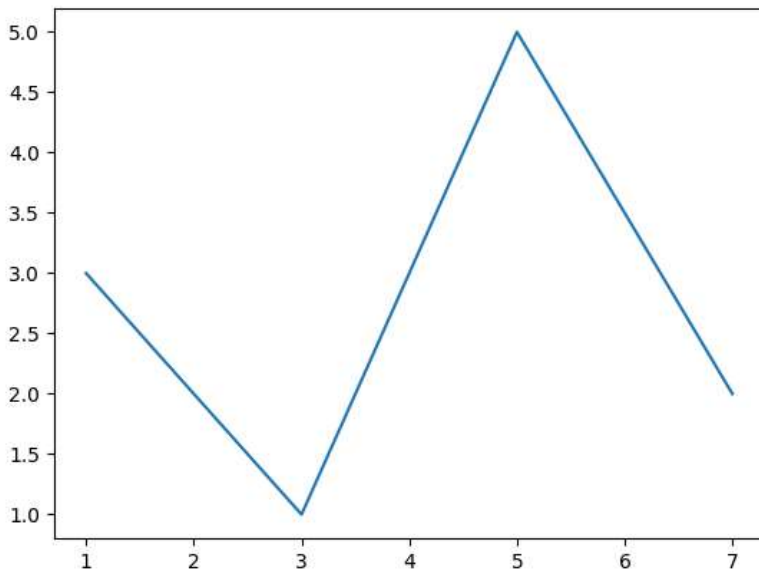
```
print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout)
```

Prints the values to a stream, or to sys.stdout by default.
 Optional keyword arguments:
 file: a file-like object (stream); defaults to the current stdout or stderr
 sep: string inserted between values, default a space
 end: string appended after the last value, default '\n'
 flush: whether to forcibly flush the stream.

▼ Data Visualization

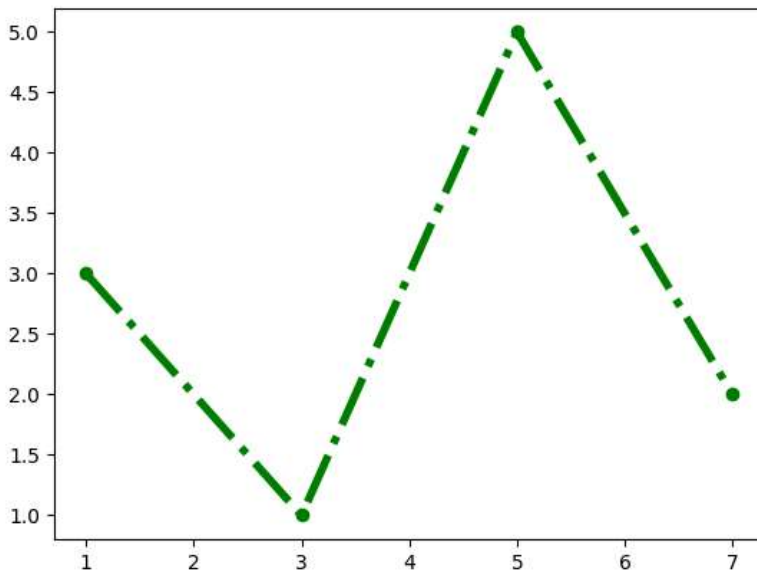
```
#LINE PLOT  
X=[1,3,5,7]  
Y=[3,1,5,2]  
import matplotlib.pyplot as plt  
plt.plot(X,Y)
```

[<matplotlib.lines.Line2D at 0x7efcc17abdf0>]



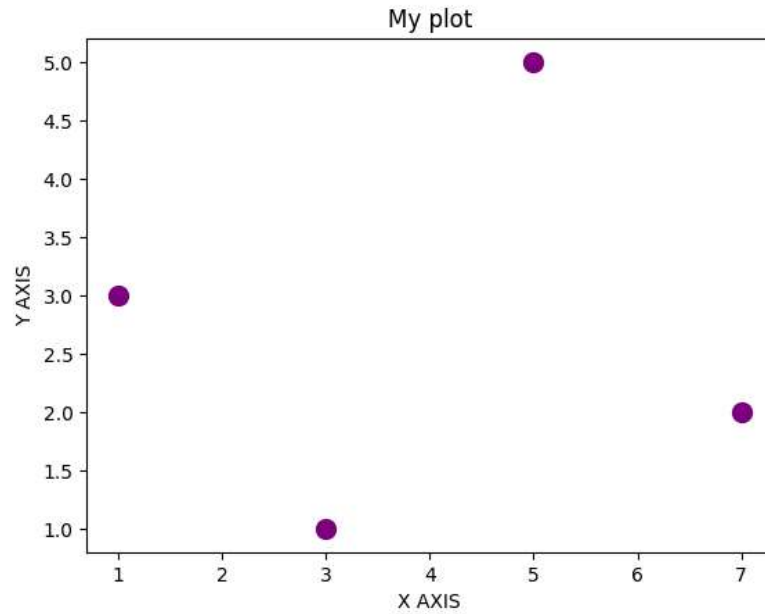
```
plt.plot(X,Y,marker='o',color='green',ls='dashdot',lw=4)
```

[<matplotlib.lines.Line2D at 0x7efcc04dee90>]



```
#SCATTER PLOT
plt.scatter(X,Y,s=100,color='purple')
plt.xlabel('X AXIS')
plt.ylabel('Y AXIS')
plt.title('My plot')
```

```
Text(0.5, 1.0, 'My plot')
```



```
%%writefile calc.py
def addition(x, y):
    return x + y
def subtraction(x, y):
    return x-y
def multiplication(x, y):
    return x * y
def division(x, y):
    return x / y

if __name__ == '__main__':      #This line checks whethe
    print("<> Running the module as a script! <>")
#This line is a conditional statement that checks whether
```

Overwriting calc.py

```
%cat calc.py

def addition(x, y):
    return x + y
def subtraction(x, y):
    return x-y
def multiplication(x, y):
    return x * y
def division(x, y):
```

```
        return x / y

if __name__ == '__main__':
    print("<> Running the module as a script! <>")

import calc
dir(calc)

['__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 'addition',
 'division',
 'multiplication',
 'subtraction']

from calc import addition
addition(8, 5)
#add(7,6)

13

import calc
calc.division(8,3)

2.6666666666666665
```